E-COMMERCE ANALYTICS · DIGITAL MARKETING INTELLIGENCE · BI

Brand Visibility Intelligence Dashboard

How brands rank, price and discount in Google Shopping search results, built from SerpApi data merged with the provided dataset, stored in SQLite and explored in Streamlit.

Author	Aryaman Bhatnagar
Data snapshot	17 Oct 2026 · Google Shopping, India
Dataset	1,421 products · 92 brands · 47 platforms · 5 keywords
Stack	Python, Requests, Pandas, SQLite, Matplotlib, Seaborn, Plotly, Streamlit
Live dashboard	*see README*
Source code	*see README*

Contents

1. Executive summary
2. Problem statement and business use cases
3. Data sources
4. Approach and architecture
5. Data extraction (API)
6. Data merging
7. Data cleaning
8. Feature engineering
9. Data storage (SQL)
10. Exploratory data analysis
11. Dashboard
12. Business insights and recommendations
13. Assumptions and limitations
14. Reproducing the project
15. Appendix A: the 30 data-cleaning questions
16. Appendix B: the 30 EDA questions

1. Executive summary

This project measures how visible brands are in online shopping search results and what drives that visibility. Product listings for five search keywords were pulled from Google Shopping through SerpApi (200 ranked results) and merged with the provided dataset (1,320 rows). After cleaning, 1,421 products remain: 195 real Google Shopping listings and 1,226 from the provided file.

1,421	89	47	₹6,854
clean products in SQLite	brands in real search results	sellers / platforms (real data)	average real price

Key findings

1. **The provided CSV is synthetic and can't support brand or ranking conclusions.** Every brand sells every category, positions are just row order, and every ranking test on it comes out at $\rho \approx 0$. Real insights come from the API data.
2. **Visibility can't be bought with price, discounts or ratings.** None of the three correlates with search position, and together with review count they predict a top-10 spot barely better than chance (AUC 0.55). For a brand, the realistic levers are presence (more listings, more sellers) and relevance (titles that match what shoppers search for), not discounting.
3. **Having more listings doesn't mean ranking higher.** Amazon carries 28% of all listings but only 20% of them reach the top 10. Pepperfry has 7 listings and 5 of them are in the top 10. A specialist seller can out-rank a big marketplace in its niche.
4. **The real market is budget-led.** 91% of products are under ₹15k and the median is ₹4,626. A launch priced above ₹15k enters a thin segment, about 9% of listings, where it is easier to stand out but has fewer buyers.
5. **Discounting is a platform habit, not a ranking strategy.** AJIO and Flipkart discount every listing, Amazon only 31% of its listings, and discounted products don't rank better ($p = 0.71$).
6. **Social proof is scarce and concentrated.** 56% of real products have no reviews, and the top 10% of reviewed products hold 56% of all reviews. For a new product, getting its first reviews is the realistic way to stand out.
7. **Brand-owned stores compete directly with marketplaces.** Puma, boAt and Ember sell through their own stores on Google Shopping, and they list the most expensive item in three of the five keywords.

Limitations: one snapshot taken on 17 Oct 2026; 40 results per keyword; India locale only; 56% of API ratings missing; small per-brand samples (3–7 listings). Treat brand-level API rankings as directional.

2. Problem statement and business use cases

Problem: build a Brand Visibility Intelligence Dashboard by extracting product data from online shopping search results through an API, and analyse how brands perform in search rankings, pricing strategies and platform competition.

Business use cases addressed

Use case	How the project answers it
Competitive analysis for e-commerce brands	Brand listing counts, top-10 presence and visibility score (Brands page, EDA E6–E10)
Pricing strategy optimisation	Price distribution, price bands, price index vs category, price vs rank (Pricing page, E11–E15)
Market positioning for new launches	Share of products per price band and how crowded each band is (E12)
Platform performance comparison	Listings, prices, ratings and ranking per platform (Platforms page, E21–E25)
Identifying high-visibility brands and products	Visibility score, top-10 products (Visibility page, E26–E30)
Monitoring discount strategies	Discount share and depth by brand and platform (E16–E20); the pipeline re-runs on demand to track changes

3. Data sources

3.1 Google Shopping via SerpApi (real data)

Engine `google_shopping`, Google domain `google.co.in`, `gl=in`, `hl=en`, location India, so prices come back in rupees like the provided file. The same five keywords as the CSV were queried: *microwave oven convection*, *air fryer India*, *power bank fast charging*, *office chair ergonomic*, *men running shoes*. Each search returns one page of 40 ranked products; SerpApi ignores pagination for this engine. Raw JSON responses are cached in `data/raw/serpapi/`.

3.2 Provided dataset (`brand_dirty_dataset.csv`)

1,320 rows and 7 columns (`keyword, title, price, rating, reviews, platform, delivery`) with planted problems: placeholder text, a -999 price code, mixed-case platform names, "!!!" in titles, duplicates and extreme prices. It turned out to be synthetic: each of its 9 brands sells every product category, and it has no search positions.

3.3 Variables

Column	Type	Description
keyword	text	Search term used to find the product
title	text	Product name as listed
price	float (₹)	Selling price after any discount
raw_price	float (₹)	Original price before discount (API `old_price`; equals price when none is shown)
rating	float	Average user rating out of 5
reviews	integer	Number of customer reviews
platform	category	Seller or marketplace (Amazon, Flipkart, Croma...)
delivery	category	Delivery terms (Free Delivery, N Days, Unknown)
position	integer	Rank in the search results (1 = top); row order for the CSV
data_source	category	`api` or `csv`

3.4 Products per keyword after cleaning

Keyword	API products	CSV products	API avg price (₹)	API % discounted
air fryer india	40	240	5,611	60.0
men running shoes	40	244	2,620	37.5
microwave oven convection	40	262	15,744	55.0
office chair ergonomic	35	237	8,947	31.4
power bank fast charging	40	243	1,608	45.0

4. Approach and architecture

The work follows the nine steps in the brief. Each step is a separate module, and every rule (keywords, thresholds, bands, aliases) lives in `config.py` rather than in the code.

#	Step	Implementation
1	Data extraction	`src/extract.py` : SerpApi requests, JSON cache, flattening to a DataFrame
2	Additional dataset	`src/load.py` : reads the CSV as text so dirty values survive until cleaning
3	Data merging	`load.merge` : aligned columns and `pd.concat` , with a `data_source` tag
4	Data cleaning	`src/clean.py` : 8 ordered steps, each logged
5	Feature engineering	`src/features.py` : brand, visibility score, bands, discount
6	Data storage	`src/db.py` : SQLite tables and views
7	EDA	`notebooks/eda.ipynb` : 60 questions with code, output and written answers
8	Visualisation	`app/` : Streamlit dashboard reading SQLite through parameterised SQL
9	Insight generation	Notebook “Key insights”, dashboard insight cards, sections 10–12 of this report

`python pipeline.py` runs steps 1–6 end to end in a few seconds. Add `--refresh-api` to fetch fresh results (5 API searches), which makes repeat monitoring a one-line job.

5. Data extraction (API)

For every keyword, `fetch_keyword` calls `https://serpapi.com/search.json` with the parameters above. Errors are raised with a clear message and without the request URL, which contains the API key. The key is read from a `.env` file that git ignores. Each product in `shopping_results` becomes one row with title, price, original price, rating, reviews, seller, delivery and position.

Problems found in the API data, and the fixes

Problem	Fix
For strings like `"60% off₹990"`, SerpApi's `extracted_old_price` returns **60** (the percentage), not 990	The ₹ amount is read from the `old_price` text; a "N% off" tag is converted back to a price
Installment listings: `extracted_price` is the monthly amount (the brief's own sample shows $29.12/mo)	Price = monthly amount × number of periods
`categorized_shopping_results` carousels ("Deals", "Under ₹7,000") restart their positions at 1	Only the main ranked list is used, so every product has a unique, comparable rank (1–40)
Seller names vary (`Amazon.in`, `Myntra – MNow`, `Bata IN`)	Suffixes are removed and names are mapped to one platform name

Result: 200 API rows (40 per keyword), saved as `data/processed/api_products.csv`.

6. Data merging

The CSV is loaded with every column as text, its columns are lower-cased to match, and two columns the CSV lacks are added: `position` (row order within keyword) and `raw_price` (empty). Both sources then share the same columns, and `pd.concat` appends them into one 1,520-row dataset. Each row keeps a `data_source` tag because the two sources behave very differently: CSV prices are about 4× real prices, and only the API has real ranks and original prices.

7. Data cleaning

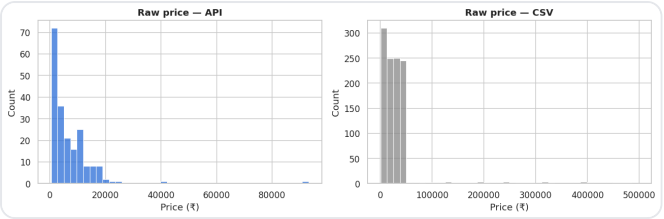
Cleaning runs as eight ordered functions. The rule throughout: **an impossible value becomes missing, an implausible extreme is capped, and no row is dropped without a reason**. Every filled or capped value is flagged.

Issue found	Size	Treatment
Price as text (`"Not Available"`), reviews as text (`"many"`)	66 / 26 cells	Parsed safely; placeholders become missing
Price of -999	26	Invalid code → missing
Missing price / rating / reviews	see C4–C5	Median for the same keyword *and* source, flagged in `*_imputed` . API products with no review count get 0 reviews
Extreme CSV prices (up to ₹4.98 lakh)	46 capped	Capped at per-keyword Q3 + 1.5×IQR; real API prices left as they are
“!!!” and stray symbols in titles	136 + 4	Removed, spaces collapsed, brand capitalisation kept
Platform spelt 61 ways	→ 47	Suffixes stripped, aliases mapped
Duplicate listings	99	Same keyword + title + platform + price after cleaning; first kept
Delivery in 9 formats	→ 4	Free Delivery / N Days / Paid Delivery / Unknown

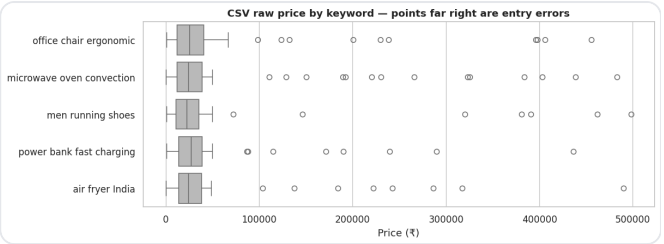
Cleaning log (stored in the `cleaning_log` table)

#	Step	Rows before	Rows after	Missing before	Missing after
1	standardize_columns	1520	1520	4316	4316
2	convert_numeric	1520	1520	4316	4665
3	invalidate_bad_values	1520	1520	4665	4691
4	clean_text	1520	1520	4691	4691
5	standardize_categories	1520	1520	4691	4670
6	remove_duplicates	1520	1421	4670	4349
7	cap_price_outliers	1421	1421	4349	4349
8	impute_missing	1421	1421	4349	2452

The cells still missing after the last step are `product_id` / `product_link` , which only API rows have; they are not part of the analysis table. After cleaning there are no missing values, no non-positive prices, no ratings outside 0–5 and no duplicates (C26).



Raw price distribution by source, before cleaning



CSV raw prices by keyword: the far-right points are entry errors

8. Feature engineering

Feature	Definition	Why
`brand`	Known brand anywhere in the title, else the first word; generic or size words → Unbranded	Neither source has a brand column
`visibility_score`	100 / $\log_2$(position + 1): 100 at #1, 63 at #2, 29 at #10	Decays like click-through, so the top spots count most
`is_top_10`	position ≤ 10	First-screen presence
`price_range`	Budget < ₹5k · Mid-range ₹5k–15k · Premium ₹15k–35k · Luxury ₹35k+	Positioning bands
`rating_range`	Below 3 · 3–4 · 4–4.5 · 4.5–5	Dashboard filter
`review_engagement`	No reviews · 1–99 · 100–999 · 1,000+	Low vs high engagement
`discount_amount`, `discount_pct`, `has_discount`	raw_price − price, and as % of raw_price	Offer analysis
`*_imputed`, `price_capped`	1 where a value was filled or capped	Lets every analysis exclude filled values

9. Data storage (SQL)

Python connects to SQLite with `sqlite3`. `pipeline.py` rebuilds the database from scratch each run. Constraints guard the data: prices must be positive, ratings between 0 and 5, and the source either api or csv.

- **products**: the clean analysis table (1,421 rows, 23 columns: the variables in section 3 plus the features in section 8).
- **raw_products**: the merged rows exactly as extracted (1,520 rows), for traceability.
- **cleaning_log**: the step-by-step log shown in section 7.
- **Views** `brand_summary`, `platform_summary` and `keyword_summary` pre-aggregate per brand, platform and keyword for quick queries.

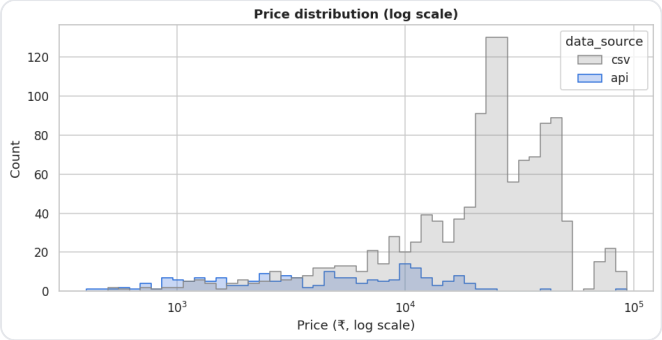
The dashboard never loads a CSV: each sidebar filter is added to a parameterised `WHERE` clause, for example `WHERE data_source IN (?) AND keyword IN (?)`, and every chart is its own SQL query.

10. Exploratory data analysis

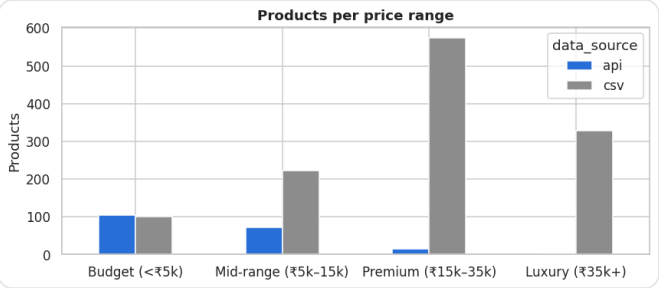
All 30 EDA questions are answered in the notebook and in Appendix B. The highlights by theme are below. Ranking conclusions use the real Google Shopping data, since the CSV's positions are only row order.

General market

Right-skewed. **Half of real products cost under ₹4,626 and 90% under ₹14,860**: a mass market with a thin premium tail. CSV prices are spread evenly from about ₹6k to ₹46k (10th–90th percentile), which doesn't resemble real prices for power banks or shoes.



Price distribution by source (log scale)

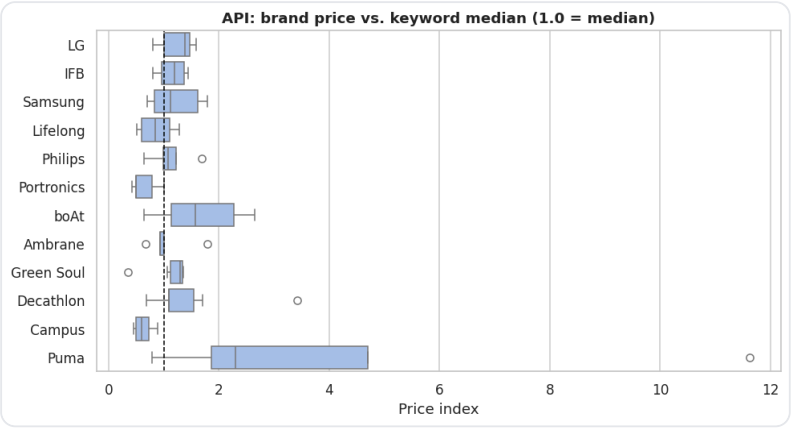


Products per price range by source

Brands

Overall: **LG (159), HP (157), Philips (144)**. These counts come from the CSV, where 9 brands share about 1,200 rows almost equally. **In real search results no brand dominates:** Decathlon and LG lead with only 7 listings each out of 195, followed by boAt, Green Soul, IFB and Samsung with 6.

The chart compares each brand's prices with the median for its keyword (1.0 = typical). **Puma (2.3×) and boAt (1.6×) price well above their categories.** LG (1.4×) and IFB (1.2×) are the premium microwave brands, while **Portronics (0.5×) and Campus (0.6×) compete on price.** The index removes the category effect: without it, any microwave brand would look expensive next to any shoe brand.



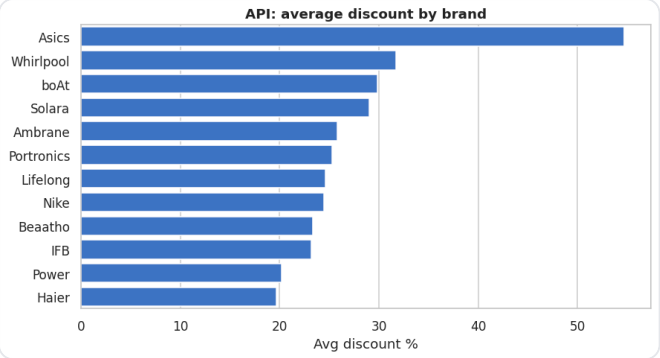
Brand price relative to its keyword median (1.0 = typical)

Pricing and discounts

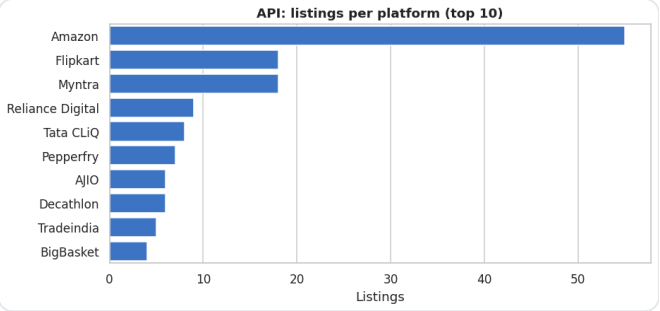
No. Across all keywords, price and position are unrelated (ρ = +0.05, p = 0.51). Within keywords, only microwaves show a hint (ρ = +0.31, p = 0.05, i.e. pricier microwaves rank slightly lower), and with five tests one borderline result is expected by chance. Google neither rewards nor penalises price as such.

46% of real listings show a discount (90 of 195), averaging **31% off** when they do. The CSV shows 0%, because it has no original price. Almost half of real listings carry a strike-through price.

AJIO (51%) and Flipkart (36%) show a discount on every listing, and Myntra averages 35%. Amazon averages only 5% (31% of its listings discounted), and Decathlon and Pepperfry show none. Fashion marketplaces compete on discounts; Amazon shows strike-through prices far less often.



Average discount by brand (API)



Listings per platform (API, top 10)

Platforms

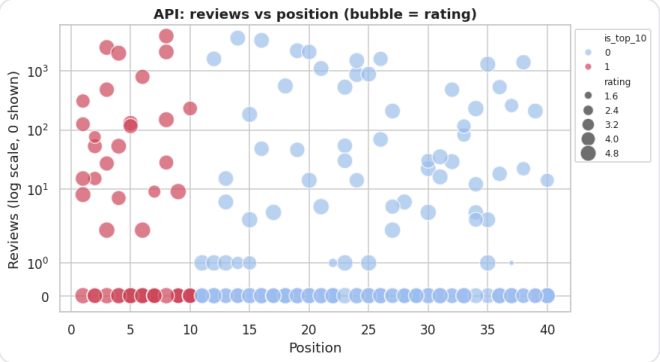
API: **Pepperfry ranks best** (average position 10.6, with 5 of its 7 listings in the top 10) because Google puts its office chairs at the top. **Amazon has by far the most listings but averages position 21.7, with only 20% in the top 10.** Having more listings doesn't mean ranking higher. CSV platforms all average about 123, so there's no signal there.

Visibility and ranking

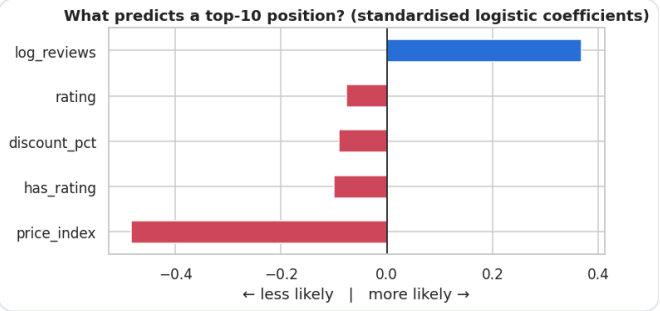
None. ρ = 0.00 in the API (p = 1.0) and −0.02 in the CSV. Google Shopping doesn't rank higher-rated products higher.

Barely. Products in the top 10 are a little more likely to have any reviews (51% vs 42%), but the number of reviews doesn't correlate with position (ρ = −0.08, p = 0.28; among reviewed products only, −0.07). If anything, having reviews at all matters more than how many, and even that effect is small.

Top-10 products are slightly cheaper for their category (price index 0.96 vs 1.01) and slightly more often reviewed. In a logistic model, more reviews (+0.37) and a lower relative price (−0.48) point toward the top 10. **But the model barely beats a coin flip (cross-validated AUC 0.55 ± 0.04).** Price, rating, reviews and discount together explain very little of the ranking, so Google's ranking must depend mainly on things this dataset doesn't capture, probably query relevance, merchant signals and paid placement. That is itself the key finding of this section.



Reviews vs search position (API)



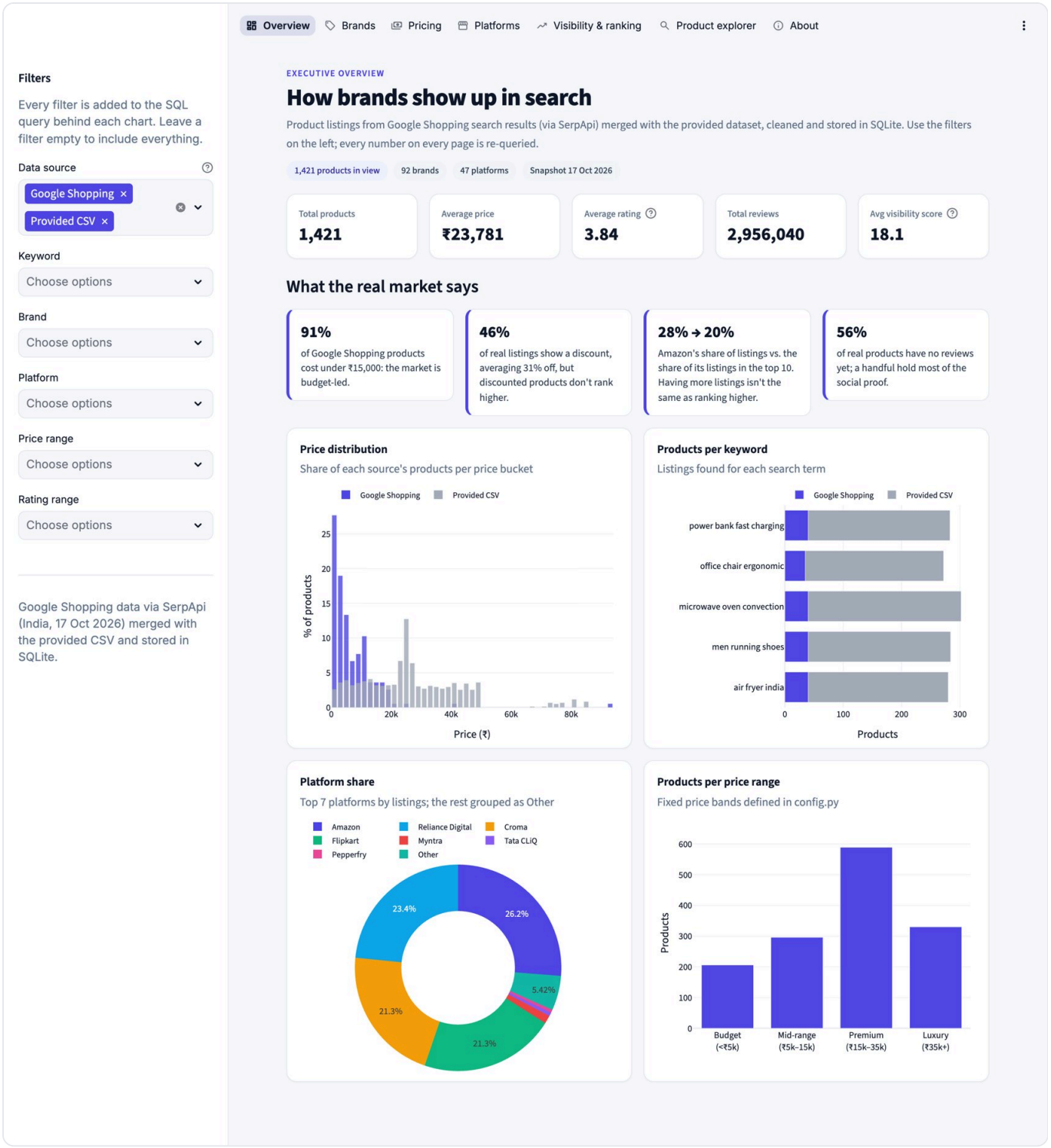
What predicts a top-10 position (logistic coefficients)

11. Dashboard

Built in Streamlit with seven pages in a top navigation bar. A sidebar of filters (data source, keyword, brand, platform, price range, rating range) is shared by every page and changes the SQL behind each element. Two rules keep the numbers honest: discount and ranking averages use Google Shopping rows only, and average ratings use real ratings only. Live version: *see README*.

11.1 Overview

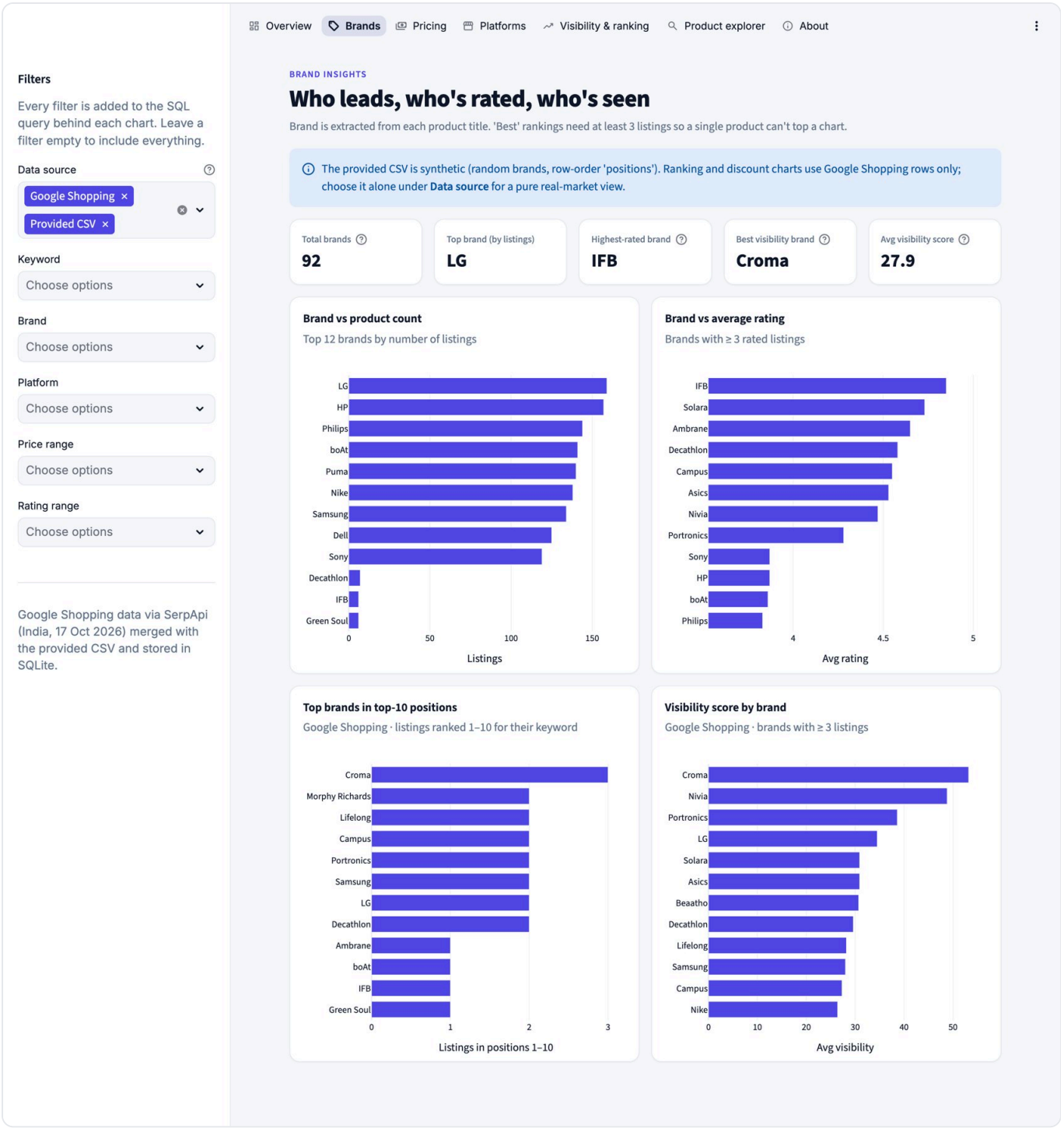
KPIs: total products, average price, average rating, total reviews, average visibility score. Real-market insight cards. Charts: price distribution (histogram), products per keyword (bar), platform share (donut), products per price range.



Overview page

11.2 Brands

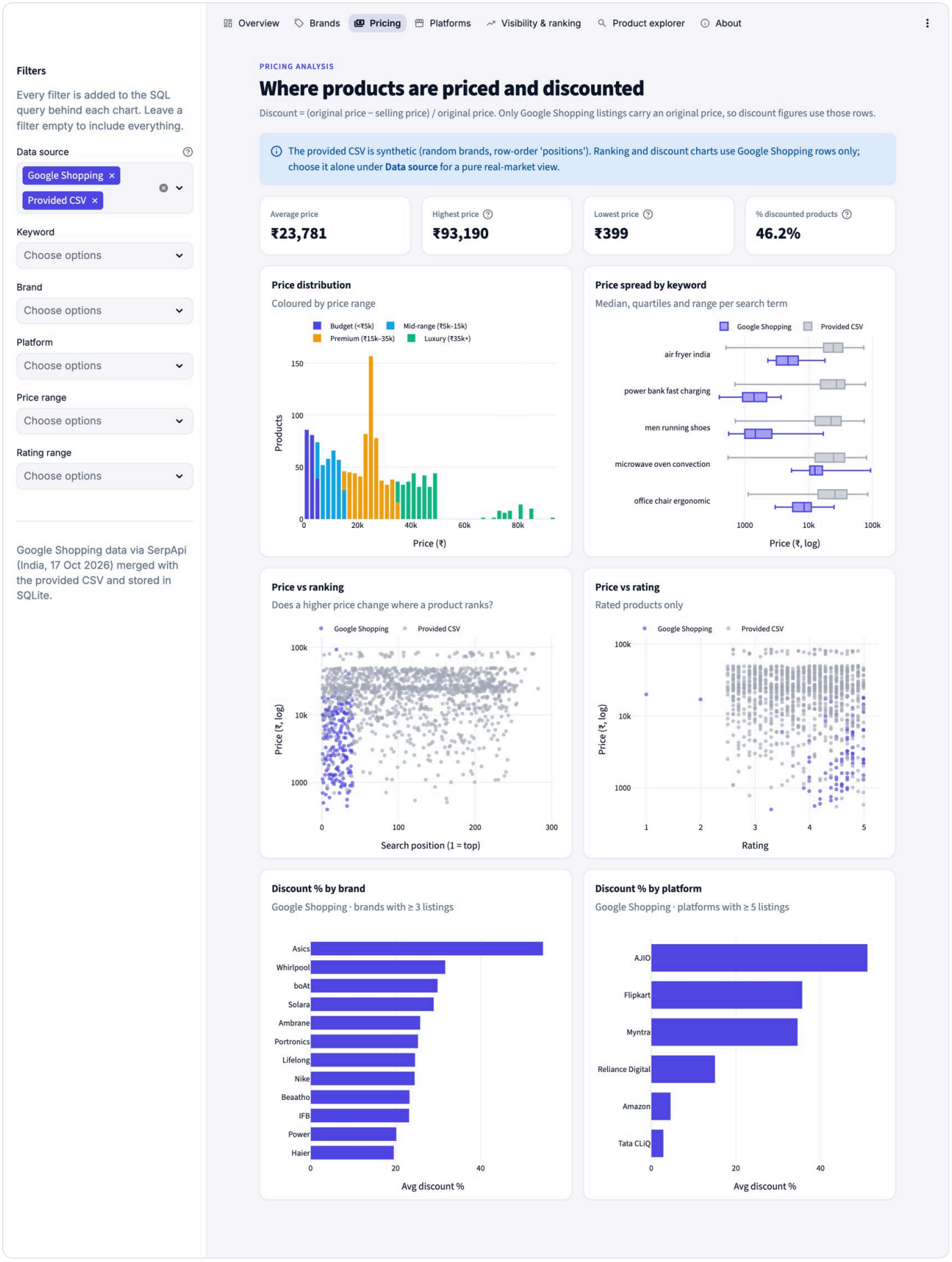
KPIs: total brands, top brand, highest-rated brand, best-visibility brand, average visibility. Charts: brand vs product count, brand vs average rating, top brands in top-10 positions, visibility score by brand.



Brands page

11.3 Pricing

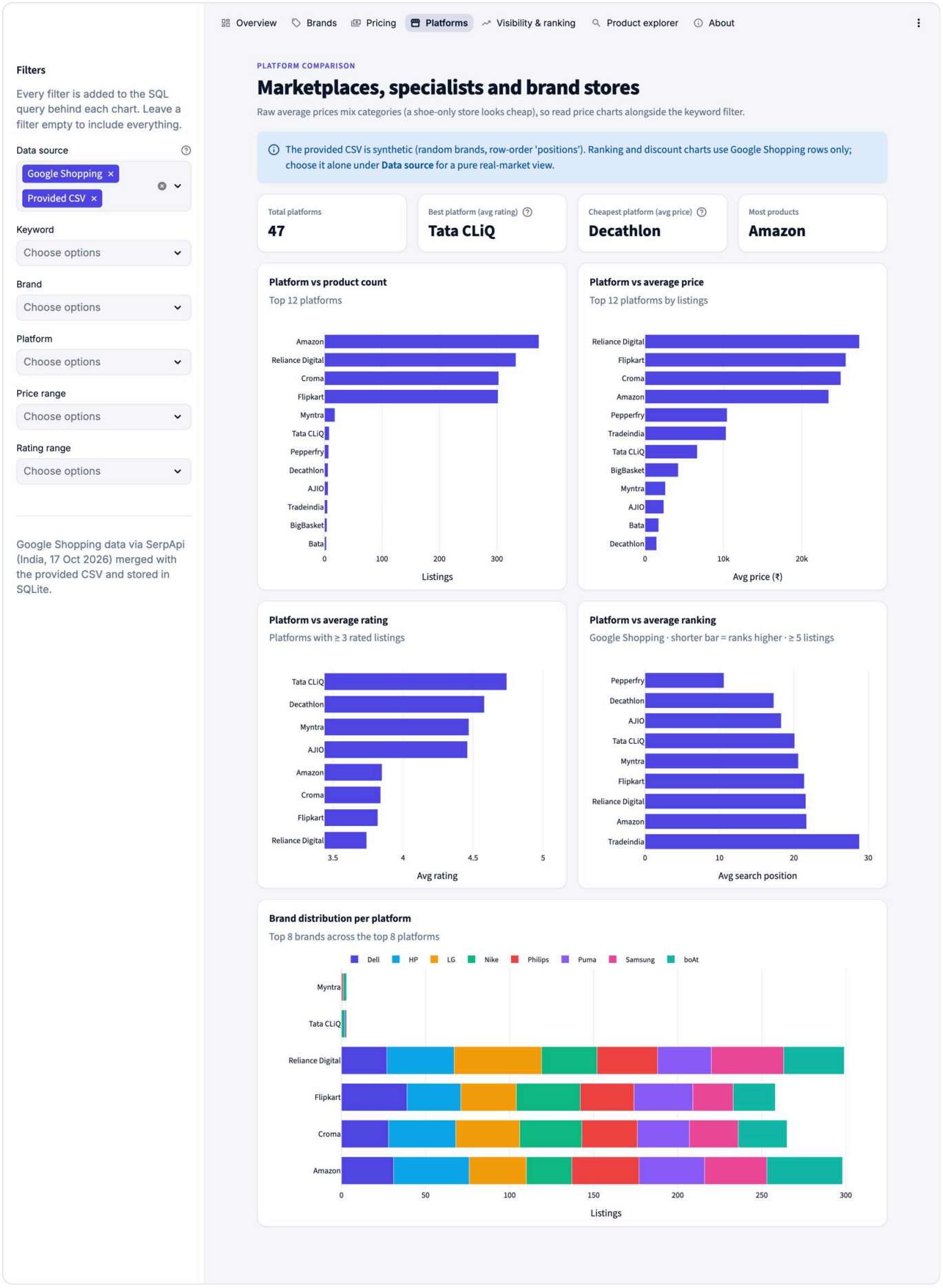
KPIs: average, highest and lowest price, % discounted products. Charts: price distribution, price spread by keyword, price vs ranking (scatter), price vs rating (scatter), discount % by brand and by platform.



Pricing page

11.4 Platforms

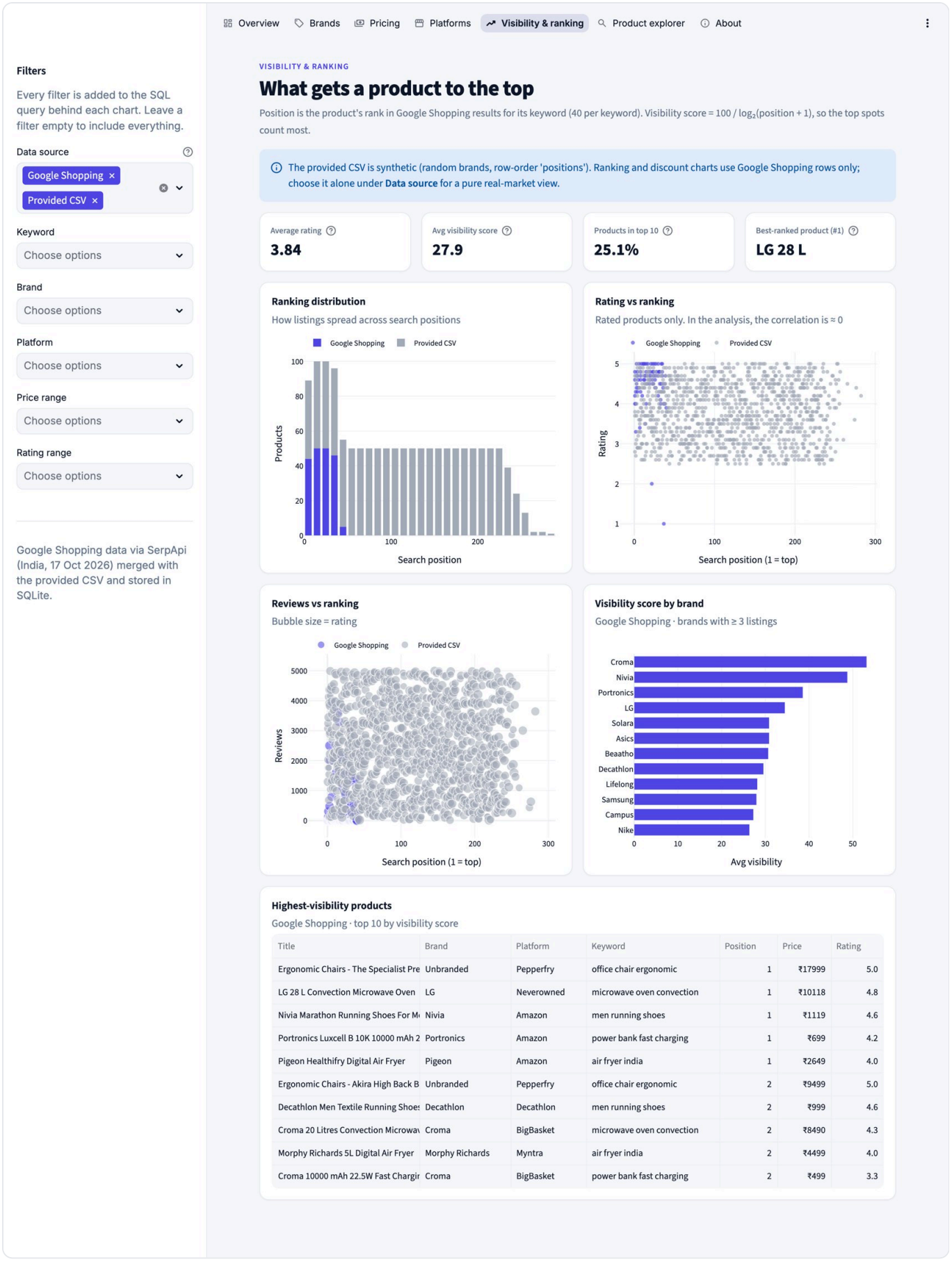
KPIs: total platforms, best platform (rating), cheapest platform, platform with most products. Charts: platform vs product count, average price, average rating, average ranking, and brand distribution per platform (stacked bar).



Platforms page

11.5 Visibility & ranking

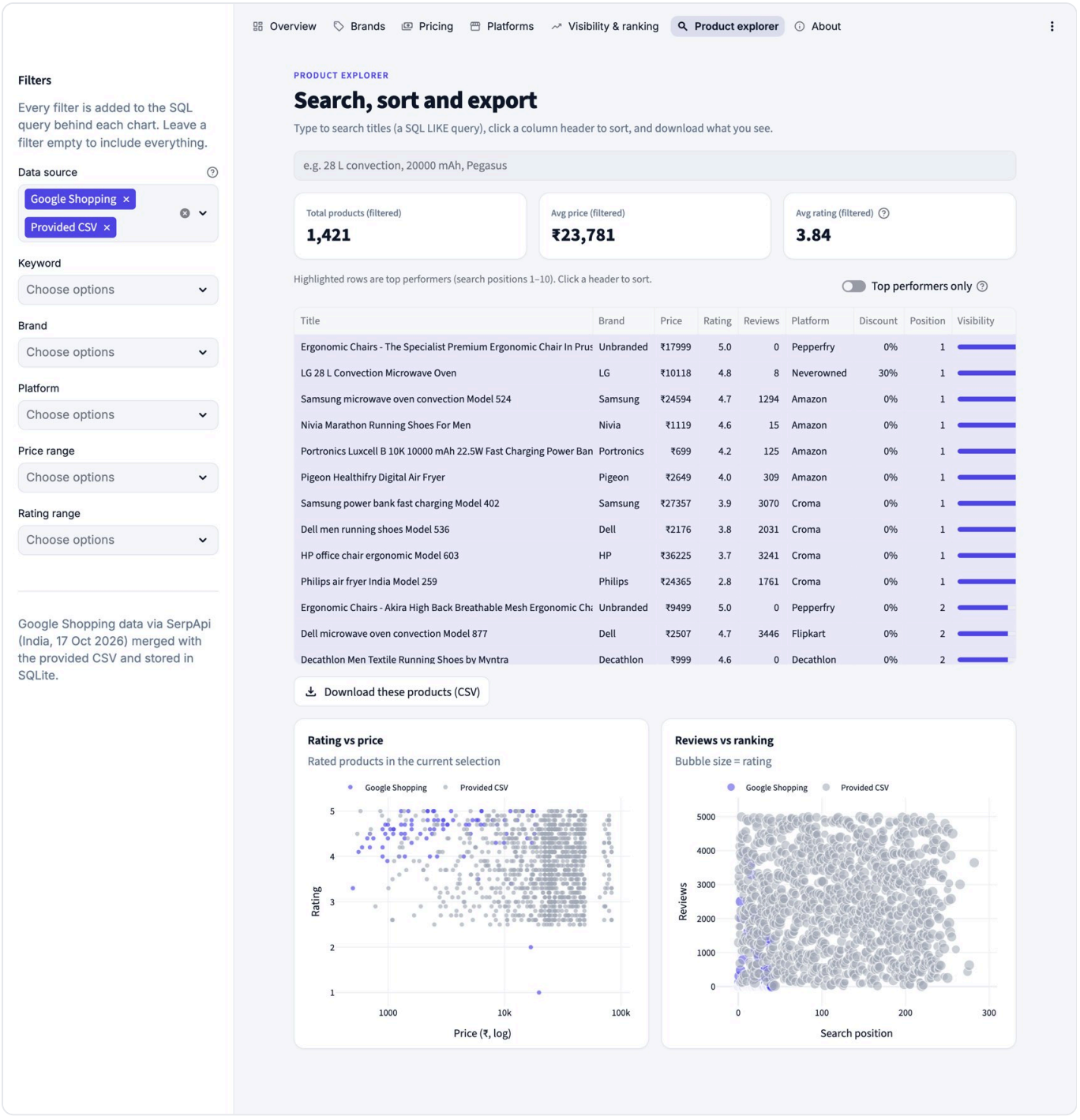
KPIs: average rating, average visibility score, % of products in the top 10, best-ranked product. Charts: ranking distribution, rating vs ranking, reviews vs ranking (bubble), visibility score by brand, and a table of the highest-visibility products.



Visibility & ranking page

11.6 Product explorer

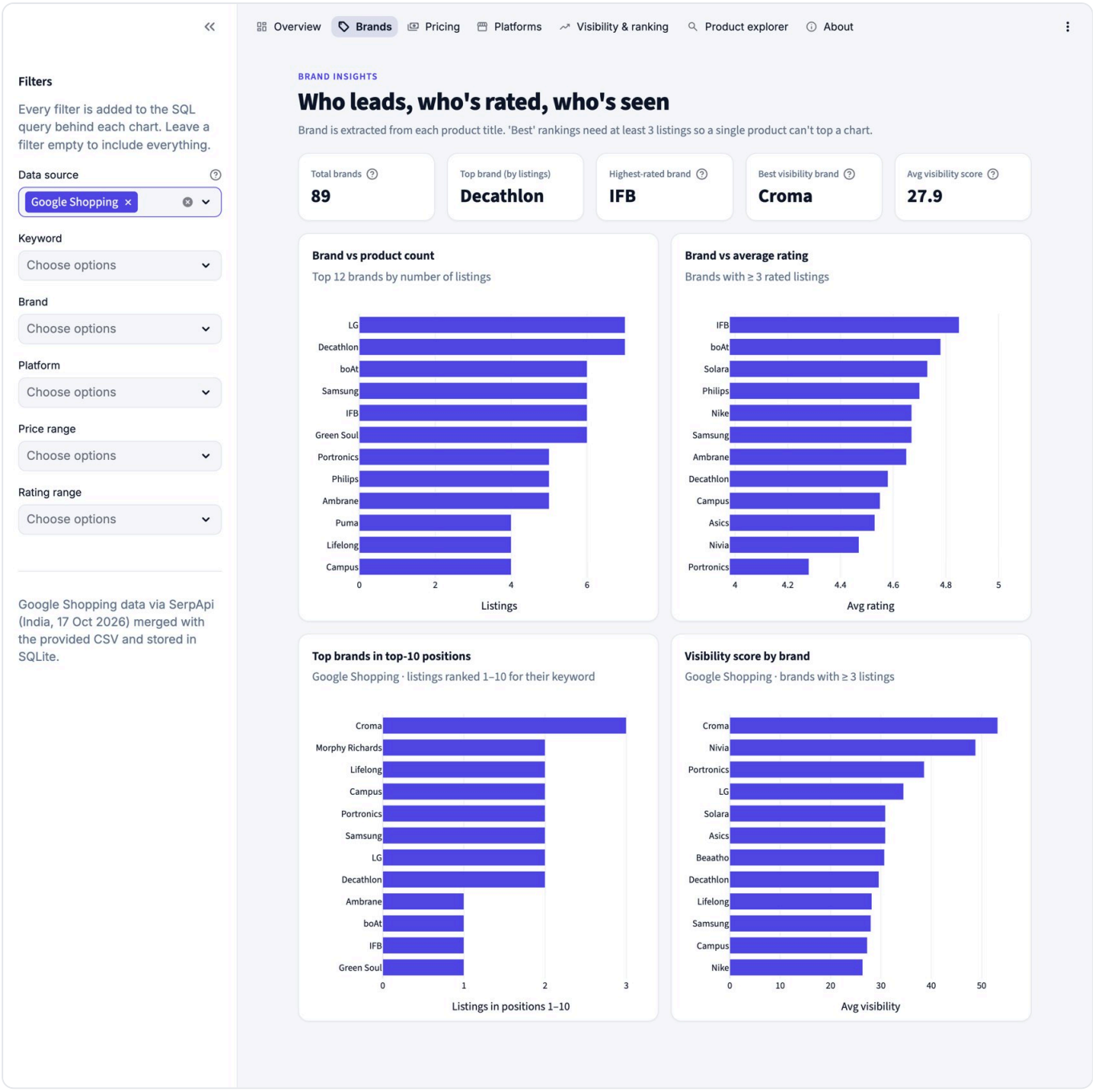
Title search (SQL LIKE), filtered KPIs, a sortable table (title, brand, price, rating, reviews, platform, discount, position, visibility) with top performers highlighted, CSV download, rating vs price and reviews vs ranking charts.



Product explorer page

11.7 Filters in action

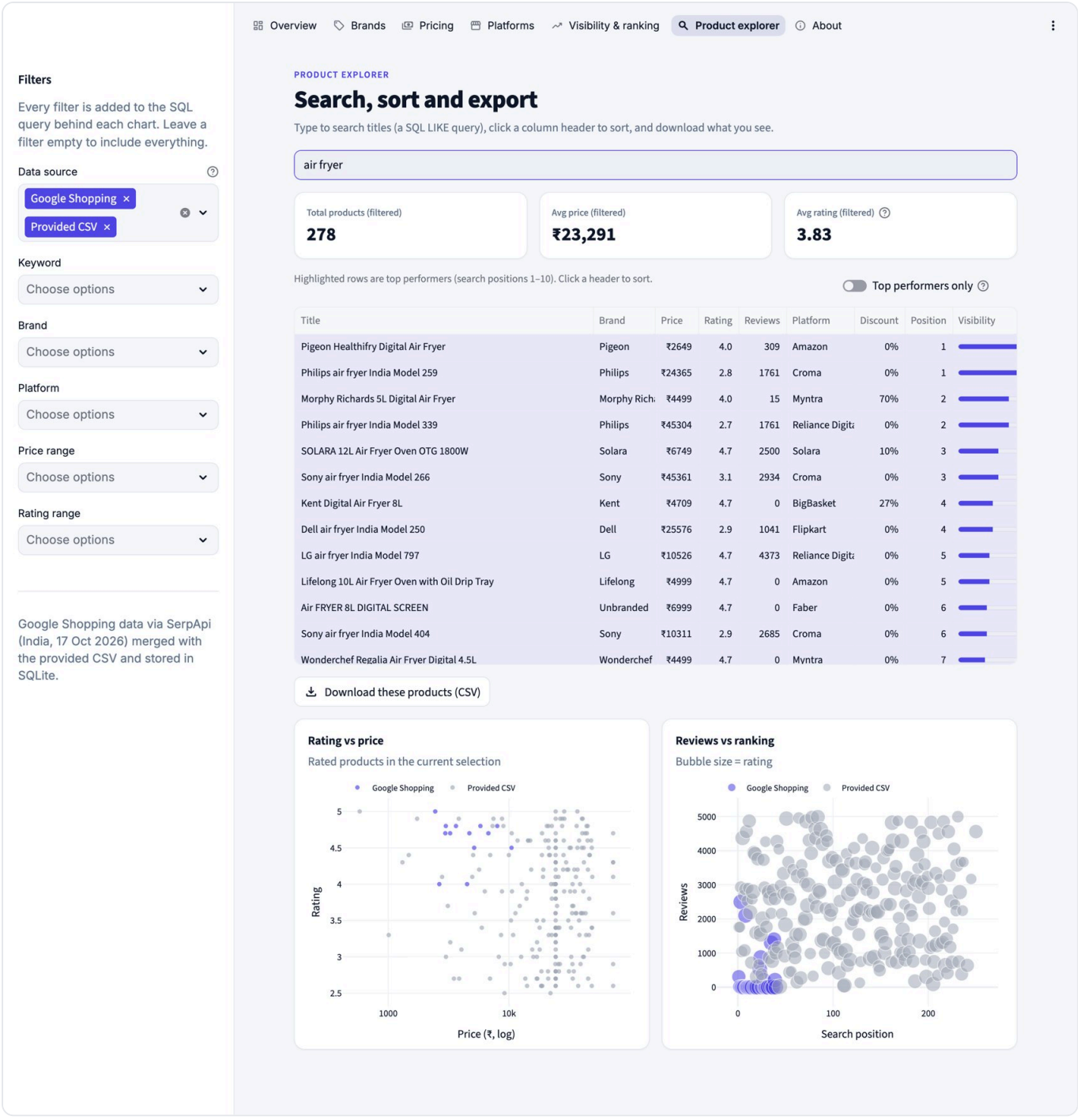
The Brands page with the data source filter set to Google Shopping only: every KPI and chart is re-queried, and the real market's brand leaders appear.



Filters in action page

11.8 Search in action

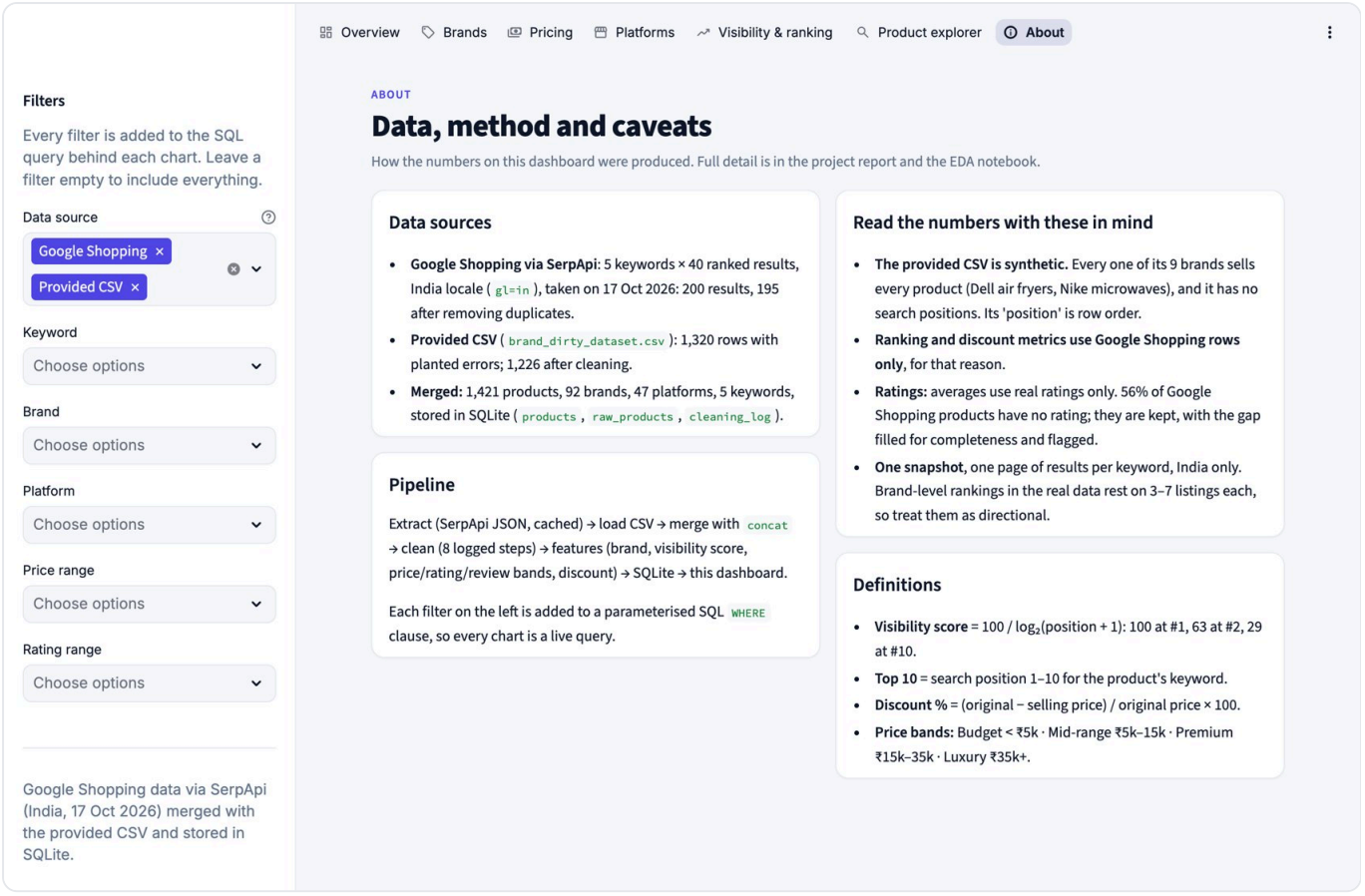
The product explorer after searching “air fryer”.



Search in action page

11.9 About

Data sources, pipeline, caveats and metric definitions, for anyone reading the dashboard cold.



About page

12. Business insights and recommendations

The seven key findings are listed in the executive summary. Here they are mapped to the business use cases in the brief.

Use case	What the data shows
Competitive analysis	No brand dominates real search results: the leaders have 7 of 195 listings. Top-10 slots are spread thinly (Croma, with 3 of 3 listings in the top 10, is the most consistent).
Pricing strategy	Price relative to category doesn't relate to rank. Puma and boAt price 1.6–2.3× their category median; Portronics and Campus compete at about 0.5×.
Launch positioning	54% of real listings are under ₹5k and 91% under ₹15k; the Premium and Luxury bands hold 9%.
Platform comparison	Amazon has the most listings (28%) but only 20% of them rank in the top 10; Pepperfry places 5 of 7 in the top 10. Specialists win their niche.
High-visibility brands	Croma, Nivia, Portronics and LG have the highest average visibility among brands with 3+ listings.
Discount monitoring	46% of real listings show a discount (31% off on average). AJIO and Flipkart discount every listing, Amazon only 31% of its listings. Discounted products don't rank better (p = 0.71).

Recommendations

1. **Compete on coverage and relevance, not discount depth.** Discounts, price and ratings don't move search rank. Brands should list with more sellers, including specialist stores that win their niche (Pepperfry for chairs, Decathlon for shoes), and write titles that match how shoppers search (brand + core spec).
2. **On this evidence, pricing above the category doesn't cost visibility.** A product's price relative to its category is unrelated to its rank (Spearman ρ = +0.05, p = 0.51), so price can be set for margin and positioning rather than to chase rank. With 195 real listings this is a weak signal, worth re-testing as more snapshots are collected.
3. **Launch where the field is thin, or go where the volume is.** 91% of real listings are under ₹15,000. A launch above that point competes with fewer products; below it, it needs to stand out on reviews and seller coverage.
4. **Treat discounts as a platform norm.** On AJIO and Flipkart every listing shows a discount, so a discount there is expected rather than a differentiator. On Amazon, an honest everyday price is the norm.
5. **Get the first reviews.** 56% of real products have none. Top-10 products are slightly more likely to have any reviews (51% vs 42%), and a first review is a cheap thing to test.
6. **Monitor continuously.** `python pipeline.py --refresh-api` re-captures all five keywords with five API calls. Scheduled weekly, it turns this snapshot into a share-of-top-10 trend per brand and platform.

13. Assumptions and limitations

- **Ranking means search position.** The brief sometimes writes “rating (ranking)”; star rating and search rank are kept as separate measures.
- **The CSV is synthetic.** Its positions are row order and its brands are random, so it is used for the cleaning exercise and for whole-dataset questions, not for brand or ranking conclusions.
- **Original prices** exist only where Google Shopping shows one; elsewhere the discount is 0.
- **Filled values** keep every product in the data but are excluded from rating averages and flagged everywhere.
- **One snapshot** (17 Oct 2026), India locale, 40 results per keyword. Brand-level results in the real data rest on 3–7 listings per brand, so they show direction, not precise rankings. Over half of real ratings are missing.
- **Ranking drivers outside the data** (query relevance, merchant quality, paid placement) probably explain most of the rank, as the weak model in E30 shows.

14. Reproducing the project

Source code: *see README*

1. `pip install -r requirements.txt`
2. Optional: copy `.env.example` to `.env` and add a SerpApi key (only needed to re-fetch)
3. `python pipeline.py` rebuilds the clean CSV and the SQLite database from the cached API data
4. `jupyter notebook notebooks/eda.ipynb` for the analysis
5. `streamlit run app/streamlit_app.py` for the dashboard

Deliverables

Deliverable	Location
Source code	`src/`, `pipeline.py`, `app/`, `notebooks/eda.ipynb`
Clean dataset	`data/processed/clean_dataset.csv`
SQL database	`data/brand_visibility.db`
Dashboard	`app/` · *see README*
Report	this document, `reports/Brand_Visibility_Intelligence_Report.pdf`
Screenshots	`reports/screenshots/`

Appendix A: the 30 data-cleaning questions

C1. What is the total number of rows and columns in the dataset?

The merged raw dataset has **1,520 rows × 8 columns**: 1,320 from the provided CSV and 200 from SerpApi (40 ranked results × 5 keywords). The CSV itself has 7 columns; `data_source` is added on merge. After cleaning and feature engineering, the database table holds **1,421 rows × 24 columns**.

C2. What are the data types of each column?

In the provided CSV only `rating` is numeric (`float64`). `price` and `reviews` are read as `object` (text). `keyword`, `title`, `platform` and `delivery` are text, as expected.

C3. Which columns are expected to be numeric but are not?

`price` and `reviews`. Both should be numbers but load as text. `rating` is already correct.

C4. Which columns contain missing values?

price (192: 126 blank + 66 "Not Available"), **rating** (244), **reviews** (139: 113 blank + 26 "many") and **delivery** (21, all API rows). `keyword`, `title` and `platform` are complete. The 26 prices of -999 are also effectively missing (see C14), but a NaN check doesn't count them.

C5. What is the percentage of missing data in each column?

Overall: rating 16.1%, price 12.6%, reviews 9.1%, delivery 1.4%. The overall figure hides a split: **the CSV's gaps are in price (14.5%)**, while **the API's are in rating and reviews (56.5% each)**, because more than half of Google Shopping listings show no rating at all. API prices are never missing.

C6. How should missing values in price, rating, and reviews be handled?

Fill them; don't drop the rows. Dropping every row with a gap would remove almost 30% of the data, and more than half of the real API rows.

- **price** → median for the same keyword *and* source (207 CSV rows; no API prices were missing).
- **rating** → median for the same keyword and source (121 CSV, 109 API).
- **reviews** → CSV: keyword median (20 rows). API: **0**, because Google Shopping leaves out the count only when a product has no reviews (109 rows).

Each filled value is flagged in a `*_imputed` column, so any analysis can exclude it. Part B's rating questions do exactly that. Medians are per source because CSV prices are about 4× API prices: a pooled median would fill real gaps with synthetic values.

C7. Should missing delivery values be filled or labeled as "Unknown"?

Label them "Unknown". All 21 are API rows where Google showed no delivery information. There's nothing to infer a value from, and filling in the most common value ("Free Delivery") would overstate free delivery. "Unknown" keeps the row and keeps the gap visible.

C8. Are there columns with incorrect data types (e.g., price as text)?

Yes. `price` is text because 66 cells say "Not Available", and `reviews` is text because 26 cells say "many". A single non-numeric value is enough for pandas to read the whole column as text.

C9. How do you convert columns like price and reviews into numeric format safely?

`clean._to_number` works in three steps: 1. trim and lower-case the value, and turn known placeholders ("not available", "many", "n/a"...) into NaN; 2. strip everything except digits, `.` and `-` (this removes ₹, commas and spaces); 3. `pd.to_numeric(errors="coerce")`, so anything still unparseable becomes NaN instead of crashing.

-999 parses fine as a *number*. It's caught by a separate check (C14), because it's a valid number but an invalid *price*.

C10. How should invalid values like "Not Available" or "many" be handled?

Convert them to NaN, then handle them like any other gap (C6). They can't be repaired: "Not Available" (66 prices) carries no price, and "many" (26 reviews) says there were a lot but not how many. Converting them first is what lets these columns become numeric.

C11. Are there duplicate rows in the dataset?

Yes, **39 exact duplicate rows**: 34 in the CSV and 5 in the API, where Google listed the same offer twice. Exact matching misses most duplicates, though (see C12).

C12. What criteria should define a duplicate record (e.g., title + price)?

Duplicate = same keyword + title + platform + price, compared *after* cleaning. Title alone is the wrong test: the same model legitimately appears on several platforms at different prices, and those are separate listings that the platform comparison needs. The example above shows why cleaning has to come first. Both rows are the same Samsung listing, but one title ends in "!!!" and the platform is spelled "amazon" in one and "AMAZON" in the other, so an exact check sees two different rows.

C13. How many duplicate records exist, and how should they be handled?

39 exact duplicates before cleaning, but **99** once titles, platforms and prices are cleaned (94 in the CSV, 5 in the API). The first occurrence is kept and the rest are dropped, leaving **1,226 CSV + 195 API = 1,421 rows**.

C14. Are there any negative or zero values in the price column?

Yes: **26 prices of exactly -999**, a common placeholder for "unknown". There are no zero prices. A product can't cost a negative amount, so these are treated as missing and filled (C6).

C15. Are there unrealistic or extreme values (outliers) in the dataset?

Yes, in the CSV. Prices reach ₹4.98 lakh against a median of ₹24,100. A ₹4.4 lakh power bank or a ₹5 lakh pair of running shoes isn't a real product. These pull the CSV's mean (₹33,600) well above its median. CSV ratings (2.5–5) and reviews (14–4,997) are within normal ranges. API prices run from ₹399 to ₹93,190; the top one is a Faber 83 L built-in microwave, which really is that expensive.

C16. How should such invalid or inconsistent values be treated?

The rule: **an impossible value becomes missing, an implausibly extreme one is capped, and nothing is deleted without a reason.** The log shows each step: placeholder text → NaN (`convert_numeric`), -999 → NaN (`invalidate_bad_values`), 99 duplicates removed, CSV outliers capped, then gaps filled and flagged. The 2,452 "missing" cells left after the last step are `product_id` / `product_link`, which only API rows have; they aren't written to the database.

C17. What is the distribution of the price column?

Strongly right-skewed in both sources (skew ≈ 5.5–5.9): most products are cheap, with a long tail of expensive ones. CSV prices centre on ₹24,800, plus a cluster of extreme errors. API prices centre on ₹4,600, which is realistic for these categories. Because of the skew, the median is the better measure of a typical price, and later charts use a log scale.

C18. Are there extreme price values compared to the majority?

Yes. In every keyword the CSV's maximum is **16–22× its median**. For men's running shoes, the median is ₹22,398 and the maximum ₹4,98,050. The extreme values sit around 10–20× normal prices, which looks like extra digits typed in by mistake.

C19. Should outliers be removed or capped? If capped, at what threshold?

Capped, not removed. The rest of each row (title, rating, platform, reviews) is valid, so dropping it would lose data for the brand and platform analysis. The threshold is **Q3 + 1.5 × IQR per keyword** (the standard boxplot fence), which works out to ₹71,700–₹83,900 depending on the keyword. That caps 46 CSV prices, each flagged in `price_capped`. **API prices are not capped:** expensive real products aren't errors. Capping would have cut a ₹12,995 Nike Pegasus down to ₹5,739.

C20. Are there unwanted characters or noise in the title column (e.g., "!!!")?

Yes: **136 CSV titles end in "!!!"**. Only 4 other titles contain unusual symbols, all API titles with stray quote marks (e.g. `Shoes'"'by Myntra`).

C21. How can you clean and standardize product titles?

`clean.clean_text` replaces any character that isn't a letter, digit, space or common product punctuation (`- & . / ( ) + , ' "`) with a space, collapses repeated spaces and trims. Brand capitalisation ("HP", "boAt") is left alone, since forcing Title Case would turn "HP" into "Hp". Keywords are lower-cased so that "air fryer India" and "air fryer india" can't split into two groups.

C22. Are there inconsistencies in text formatting across columns?

Yes. - **platform:** 61 raw spellings. The CSV has case variants; the API has domain suffixes and sub-stores ("Amazon.in", "Myntra - MNow"). - **delivery:** 9 formats that mix two different things, cost ("Free Delivery") and speed ("2 Days"). - **keyword:** consistent apart from capitalisation ("air fryer India").

C23. Are platform names consistent (e.g., "amazon", "AMAZON", "Amazon")?

No. Every CSV platform appears in two spellings: "amazon" (182) and "AMAZON" (155), "croma" and "CROMA", and so on, so 4 platforms look like 8. Uncorrected, every platform chart would show Amazon twice at roughly half its real count.

C24. How can you standardize categorical values across the dataset?

Map every raw value to one canonical name. First strip domain and region suffixes (".in", ".com", " India"). Then match known aliases case-insensitively using `config.PLATFORM_ALIASES`: anything containing "amazon" becomes Amazon, anything containing "reliance" becomes Reliance Digital, and so on. Unknown sellers keep their own name, with tidied capitalisation. The rules live in config, so supporting a new platform takes one line.

C25. Are delivery values consistent and meaningful?

Only partly. Within the CSV the values are consistent (Free Delivery / 2 Days / 5 Days), but they mix cost and speed: a "2 Days" product could also be free. **In the API, every listing that shows delivery says it's free** ("Free delivery", "Free delivery by Fri", "Free 10 min delivery"), so the field tells API products apart not at all. It's cleaned to Free Delivery / N Days / Unknown and kept for filtering, but isn't used as a driver in the analysis.

C26. After cleaning, are there any remaining missing or invalid values?

No. Every check passes: no missing cells, no zero or negative prices, no ratings outside 0–5, no negative reviews, no original price below the selling price, no discount outside 0–100%, and no duplicate listings.

C27. Are all numeric columns correctly formatted and usable?

Yes. Price, discount, rating and visibility are floats; reviews and position are integers. All ranges are plausible: price ₹399–₹93,190, discount 0–83%, position 1–40 for the API (up to 283 for the CSV's row-order positions). The SQLite schema also enforces some of this with CHECK constraints (price > 0, rating between 0 and 5).

C28. Does the dataset now make logical and business sense?

It's technically clean, but the CSV doesn't make business sense. The table shows every one of the 9 CSV brands selling every product: Dell air fryers (34), Nike microwaves (24), Puma power banks (26), HP running shoes (28). Real brands don't do this, so the CSV is synthetic, and brand findings from it describe how it was generated, not the market. Its "positions" are also just row order. **The API data does make sense:** the microwave brands are LG, IFB and Samsung, and the shoe brands are Nike, Puma and Decathlon. That's why Part B relies on API data for brand and ranking conclusions.

C29. Should products with no ratings or reviews be retained or removed?

Keep them. In the API data 56% of products have no rating. Removing them would throw out half the real market and bias every average toward established products. They stay in with the median rating filled in and `rating_imputed = 1`, and rating analyses use real ratings only.

C30. Can additional features like brand extraction from title improve the dataset?

Yes. Without it there's no brand analysis at all, because neither source has a brand column. Extraction looks for a known brand anywhere in the title ("Men's PUMA X SAYSKY..." → Puma), otherwise takes the first word, and labels titles that start with a generic word ("Ergonomic Chairs – ...") as Unbranded. It finds **89 brands in the API data** (plus 14 Unbranded listings), against the CSV's 9. Size/spec tokens ("27L...") and generic openers ("Air Fryer...") are never taken as brands, and short all-caps names keep their case (HRX, EDT). Other derived features: `visibility_score`, `price_range`, `rating_range`, `discount_pct`, `review_engagement`, `is_top_10`.

Appendix B: the 30 EDA questions

E1. How many total products are available per keyword?

Combined, each keyword has **272–302 products**: microwave oven convection the most (302), office chair ergonomic the fewest (272). Every keyword contributes 40 API results, except office chairs (35 after removing 5 duplicates). The CSV adds 237–262 per keyword, so it dominates the counts.

E2. What is the overall average price of products?

The overall average is **₹23,781** (median ₹22,658), but it blends two very different datasets: the CSV averages ₹26,473, while **real Google Shopping averages ₹6,854 (median ₹4,626)**. The real market is about 4–5× cheaper than the CSV suggests.

E3. What is the distribution of price across all products?

Right-skewed. **Half of real products cost under ₹4,626 and 90% under ₹14,860**: a mass market with a thin premium tail. CSV prices are spread evenly from about ₹6k to ₹46k (10th–90th percentile), which doesn't resemble real prices for power banks or shoes.

E4. What is the average rating across all products?

CSV: **3.79**. API: **4.49** counting only real ratings (86 products), or 4.65 including filled values. Real Google Shopping ratings are high and tightly bunched (median 4.6), so a 4.2 is below average in this market.

E5. How are reviews distributed (low vs high engagement products)?

Very unequal in real data: the top 10% of reviewed API products hold 56% of all reviews. By band, 109 of 195 API products have **no reviews**, 52 have 1–99, 20 have 100–999 and only 14 have 1,000+. A handful of products collect most of the social proof. The CSV is the opposite: 955 of its 1,226 products have 1,000+ reviews, spread evenly from 14 to 4,997, another sign that it's synthetic. The bands are fixed cut-offs (`REVIEW_BANDS` in config), so they mean the same thing for both sources.

E6. Which brand appears most frequently?

Overall: **LG (159), HP (157), Philips (144)**. These counts come from the CSV, where 9 brands share about 1,200 rows almost equally. **In real search results no brand dominates:** Decathlon and LG lead with only 7 listings each out of 195, followed by boAt, Green Soul, IFB and Samsung with 6.

E7. Which brand has the highest average visibility_score?

Among brands with at least 3 API listings: **Croma (53.2), Nivia (48.8), Portronics (38.6), LG (34.5)**. Croma is a retailer's own label, and its listings sit near the top (average position 3.7). In the CSV every brand scores 16.5–17.2, with no real difference between them, as expected from random row order.

E8. What is the average rating (ranking) per brand?

API, brands with at least 3 listings: **Croma ranks best (average position 3.7)**, followed by Nivia (14.7), Samsung (15.0) and Lifelong (15.5). Average position can only be compared within a source: CSV positions run up to 283, API positions only to 40.

E9. Which brands appear most in top 10 ratings (position <= 10)?

API: **Croma** (all 3 of its listings in the top 10), then Samsung, Portronics, Morphy Richards, Lifelong and LG with 2 each. Top-10 slots are spread thinly and no brand owns the first page. In the CSV, Dell (9) and LG (8) lead, but at 3–7% of their listings that's noise.

E10. Which brands have the highest average rating?

API (real ratings, at least 3 rated products): **IFB 4.85, boAt 4.78, Solara 4.73, Philips 4.70.** CSV: HP and Sony at 3.87; differences of 0.1 between CSV brands are noise. Each API brand has only 3–4 rated products, so treat these as indicative.

E11. What is the price distribution across different brands?

The chart compares each brand's prices with the median for its keyword (1.0 = typical). **Puma (2.3×) and boAt (1.6×) price well above their categories.** LG (1.4×) and IFB (1.2×) are the premium microwave brands, while **Portronics (0.5×) and Campus (0.6×) compete on price.** The index removes the category effect: without it, any microwave brand would look expensive next to any shoe brand.

E12. How are products distributed across price_range categories?

Real market: 54% Budget (under ₹5k), 37% Mid-range, 8% Premium, 1% Luxury. The CSV is almost the reverse, with 47% Premium and 27% Luxury. For pricing strategy, the real competition is below ₹15k, where 91% of API products sit.

E13. Do higher-priced products have better rankings (rating)?

No. Across all keywords, price and position are unrelated ($\rho = +0.05$, $p = 0.51$). Within keywords, only microwaves show a hint ($\rho = +0.31$, $p = 0.05$, i.e. pricier microwaves rank slightly lower), and with five tests one borderline result is expected by chance. Google neither rewards nor penalises price as such.

E14. What is the average price per platform?

On raw averages, Flipkart looks the most expensive (₹11,700) and Myntra the cheapest (₹2,587), but that reflects **product mix**: Myntra's results are mostly shoes, Flipkart's mostly microwaves. Compared like for like with the price index, **Amazon (0.96), Flipkart (0.93) and Myntra (0.95) all price close to the market median**, while Tata CLiQ (2.1×) and AJIO (1.5×) list pricier items.

E15. Which products have the highest price within each keyword?

Microwave: **Faber 83 L built-in, ₹93,190** (Amazon). Office chair: ASTRIDE Ergofit, ₹24,998. Air fryer: Ember Marvel air-fryer oven, ₹17,999. Shoes: Puma × SAYSKY MagMax Nitro 2, ₹16,999. Power bank: boAt EnergyShroom PB665, ₹3,699. **Three of the five sell through the brand's own store** (Ember, Puma, boAt) rather than a marketplace.

E16. What percentage of products have a non-zero discount?

46% of real listings show a discount (90 of 195), averaging **31% off** when they do. The CSV shows 0%, because it has no original price. Almost half of real listings carry a strike-through price.

E17. Do discounted products have better rankings (rating)?

No. Discounted products have a median position of 21, against 19 for the rest, and 22% reach the top 10 against 28%. That's slightly *worse*, and not significant ($p = 0.71$). A discount doesn't buy visibility on Google Shopping.

E18. Which brand offers the highest average discount?

Among brands with at least 3 API listings: **Asics (55% on average, every listing discounted)**, Whirlpool (32%), boAt (30%), Solara (29%) and Ambrane (26%). When every listing is discounted this deeply, the list price may be set high on purpose. That's a hypothesis this data can't test.

E19. Which platform provides the highest average discount?

AJIO (51%) and Flipkart (36%) show a discount on every listing, and Myntra averages 35%. Amazon averages only 5% (31% of its listings discounted), and Decathlon and Pepperfry show none. Fashion marketplaces compete on discounts; Amazon shows strike-through prices far less often.

E20. Is there a relationship between discount and rating?

Among discounted API products with real ratings, bigger discounts go with somewhat lower ratings ($\rho = -0.25$), but the result **isn't significant** ($p = 0.11$, $n = 44$). It weakly hints that deep discounts are used to move weaker products; more data is needed to say.

E21. Which platform lists the highest number of products?

API: Amazon by far, with 55 of 195 listings (28%), followed by Flipkart and Myntra with 18 each. The CSV's four platforms are evenly balanced (284–324 each). The real data has a long tail: **47 different sellers** appear in 195 listings, including brand-owned stores.

E22. Which platform has the highest average rating?

API (real ratings, at least 5 rated products): **Tata CLiQ 4.74**, Amazon 4.50, Myntra 4.47, AJIO 4.46. The differences are small and the samples thin (5–23 products). CSV: Croma 3.83 is highest and Reliance Digital 3.74 lowest, a 0.1 spread that's noise.

E23. Which platform has the lowest average price?

On raw averages, Decathlon is cheapest in the API (₹1,466) and Croma in the CSV (₹25,118). But Decathlon only sells shoes. Compared like for like with the price index, **Tradeindia (0.83), Flipkart (0.93), Myntra (0.95) and Amazon (0.96) are the cheapest**, and Tata CLiQ (2.11) the most expensive. The CSV platforms are identical by construction (index 1.00).

E24. What is the average rating (ranking) per platform?

API: **Pepperfry ranks best** (average position 10.6, with 5 of its 7 listings in the top 10) because Google puts its office chairs at the top. **Amazon has by far the most listings but averages position 21.7, with only 20% in the top 10.** Having more listings doesn't mean ranking higher. CSV platforms all average about 123, so there's no signal there.

E25. Which brands are most frequently listed on each platform?

Real platforms specialise. Flipkart's results are mostly microwave brands (IFB 5, Whirlpool 3, Samsung 2). Tata CLiQ's and AJIO's are shoe brands (Nike, Adidas, Asics, Decathlon). Myntra's are shoes plus power banks (Ambrane, HRX, Portronics). In the CSV, every platform carries the same 9 brands in a different order.

E26. What is the distribution of rating (ranking)?

API positions are spread evenly from 1 to 40 within each keyword, because each search returns one page of 40. Star ratings: the CSV is spread evenly from 2.5 to 5, while the API bunches at 4.4–4.8, with a spike at the filled-in median.

E27. Which products have the highest visibility_score?

Position 1 in each keyword scores 100: **Pigeon Healthifry air fryer, Nivia Marathon shoes, LG 28 L microwave, Pepperfry's "The Specialist" chair, and Portronics Luxcell B 10K power bank.** Three of the five top spots are Amazon listings, and three cost under ₹3,000.

E28. Is there a relationship between rating and rating (ranking)?

None. ρ = 0.00 in the API (p = 1.0) and −0.02 in the CSV. Google Shopping doesn't rank higher-rated products higher.

E29. Do products with higher reviews rank better?

Barely. Products in the top 10 are a little more likely to have any reviews (51% vs 42%), but the number of reviews doesn't correlate with position (ρ = −0.08, p = 0.28; among reviewed products only, −0.07). If anything, having reviews at all matters more than how many, and even that effect is small.

E30. Which combination of factors (price, rating, reviews) influences top-ranking products?

Top-10 products are slightly cheaper for their category (price index 0.96 vs 1.01) and slightly more often reviewed. In a logistic model, more reviews (+0.37) and a lower relative price (−0.48) point toward the top 10. **But the model barely beats a coin flip (cross-validated AUC 0.55 ± 0.04).** Price, rating, reviews and discount together explain very little of the ranking, so Google's ranking must depend mainly on things this dataset doesn't capture, probably query relevance, merchant signals and paid placement. That is itself the key finding of this section.